



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Python para Ciencia de Datos

Tema 2 **Elementos fundamentales de Python**

Máster F.P. En IA & Big Data ANALYTICS
Curso 2026-2027

UPV

Resultados de aprendizaje

- Definir los conceptos de valor, variable, expresión y sentencia, y explicar sus características en Python
- Describir el concepto de tipo de datos, e identificar los tipos de datos básicos de Python (enteros, reales, cadenas y booleanos)
- Identificar los principales operadores de cada tipo de datos, así como su precedencia a la hora de construir expresiones
- Explicar el concepto de conversión de tipos y diferenciar entre la conversión implícita y explícita. Ser capaz de convertir valores entre tipos diferentes
- Utilizar correctamente las sentencias de entrada y salida en Python, incluyendo el formateo sofisticado de expresiones en sentencias de salida

Índice



- 1. Valores y variables**
- 2. Tipos, operadores y expresiones**
- 3. Tipos de datos básicos: enteros, reales, cadenas y booleanos**
- 4. Conversión de tipos**
- 5. Formatos de entrada y salida**

Índice



- 1. Valores y variables**
- 2. Tipos, operadores y expresiones**
- 3. Tipos de datos básicos: enteros, reales, cadenas y booleanos**
- 4. Conversión de tipos**
- 5. Formatos de entrada y salida**

Valores y variables

Lenguaje Python

- Como en cualquier lenguaje, en **Python** tenemos:
 - **Vocabulario** (palabras): Variables; Constantes; Funciones y operadores; Expresiones; Palabras reservadas.
 - **Estructuras** para hacer frases (sentencias) con estas palabras:
 - Patrones de sintaxis válidos para crear sentencias.
 - **Estructuras** para combinar frases (unir sentencias) y ser capaces de crear un programa que resuelva un problema:
 - Estructuras secuenciales
 - Estructuras condicionales (p. ej. if-then)
 - Estructuras de repetición (p. ej. while, for)
 - Funciones

Valores y variables

Variables

- Una **variable** es una posición de memoria que permite almacenar un **valor** y referenciarlo por un **nombre** decidido por el programador
- En Python, para poder usar una variable **no** es necesario **declararla** a priori, ni definir su **tipo de datos**
- Para **crear** una nueva variable, simplemente se le **asigna** un valor, por ejemplo:

```
longitud = 50
```

← Sentencia de **asignación**

- El **tipo de datos** de una variable coincide con el del valor (*objeto*) que almacena en un momento dado (tipado dinámico)

Valores y variables

Variables

- En Python, el **nombre** de una variable:
 - Es una secuencia de letras, dígitos y guiones bajos (`_`)
 - No se admiten otros símbolos
 - El primer carácter no puede ser un dígito
 - No puede coincidir con ninguna de las **palabras reservadas** de Python
 - Diferencia entre minúsculas y mayúsculas (`x` y `X` son distintas variables)
- Algunos **ejemplos** de nombres de variables:
 - Correctos: `ab`, `AB_`, `ABC12`, `una_variable`
 - Incorrectos: `2name`, `if`, `x.4`, `$variable`

Valores y variables

Variables

- Se suelen seguir ciertas **reglas de estilo** para definir los nombres de variables:
 - Deben comenzar con una letra minúscula, seguida de otras letras (mayúsculas o minúsculas) y un dígito
 - El subrayado (`_`) se utiliza para delimitar palabras en variables compuestas por varias palabras
 - Ejemplos: `i`, `x`, `y2`, `total`, `fecha_nacimiento`
- Es siempre mejor usar nombres de variables que tengan sentido:

```
a = 5  
b = 10  
c = a * d
```

vs

```
unidades = 5  
precio = 10  
total = unidades * precio
```

Valores y variables

Variables

- Para mostrar los mensajes en la pantalla, se utiliza la función **print()**
- Ejemplo:

```
unidades = 5  
precio = 10  
total = unidades * precio  
  
print(total)
```

Salida:

5

Valores y variables

Palabras reservadas de Python

- Son **palabras clave** que forman parte de la sintaxis del lenguaje
- Están “**reservadas**” porque al tener un significado especial para el intérprete no pueden utilizarse como **identificadores**
- Un identificador es un nombre de variable, función, clase, etc.

La lista completa la tienes en:

https://docs.python.org/es/3/reference/lexical_analysis.html#keywords



Valores y variables

Constantes

- Una variable se considera una **constante** cuando su valor es conceptualmente inmutable y por tanto no va a cambiar en toda la ejecución del programa
- La convención es nombrarlas todo en **mayúsculas** y usar el subrayado como separador de palabras
- Ejemplos:
 - `PI = 3.1415926`
 - `LIGHT_SPEED = 299792458`
 - `NAVY_BLUE_RGB = (0, 0, 128)`

Índice



- 1. Valores y variables**
- 2. Tipos, operadores y expresiones**
- 3. Tipos de datos básicos: enteros, reales, cadenas y booleanos**
- 4. Conversión de tipos**
- 5. Formatos de entrada y salida**

Tipos, operadores y expresiones

Tipos de datos

- Al programar es muy importante tener en cuenta qué tipo de datos está procesando el programa en cada expresión o sentencia
- Un **tipo de datos** está definido por:
 - Un **conjunto de valores**, y
 - Una serie de **operaciones** definidas sobre ellos
- El lenguaje Python permite trabajar con multitud de tipos de datos:
 - Básicos: enteros, reales, booleanos, cadenas, ...
 - Estructurados: lista, tupla, rango, diccionario, conjunto, ...

Tipos, operadores y expresiones

Algunos tipos de datos básicos

Tipo de dato	Valores	Operadores	Ejemplos
int	Números enteros	+ - * // % **	99 12 2147483647
float	Números reales	+ - * / **	3.14 2.5 6.022e23
bool	True/False	and or not == != > < >= <=	True False
str	Cadenas de caracteres (<i>strings</i>)	+ *	'Hola' '2.5' '¿Dónde?'

Tipos, operadores y expresiones

Operadores aritméticos (números enteros y reales)

Operador	Operación	Descripción	Ejemplo	Resultado
+ , -	Signo	Cambio de signo	-7	-7
+	Suma	Suma de números reales o enteros	7 + 3	10
-	Resta	Resta de números reales o enteros	7 - 3	4
*	Multiplicación	Multiplicación de números reales o enteros	7 * 3	21
//	División entera	División de números enteros, descartando la parte fraccionaria	7 // 3	2
%	Módulo	Módulo (o resto) de la división entera	7 % 3	1
/	División	División de números reales	7.0 / 3.0	2.33333333
**	Potencia	Elevar la base al exponente	7 ** 3	343

Tipos, operadores y expresiones

Función `type()`

- Python conoce el "tipo" (*clase*) de cualquier valor o variable, y este puede consultarse mediante la función **`type()`**

```
type('Hello')  
<class 'str'>
```

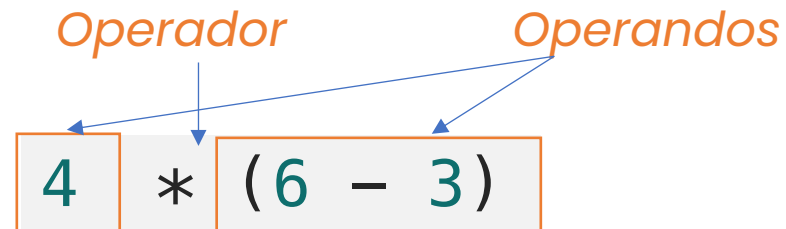
- Es importante saber qué operaciones están definidas sobre cada tipo, para evitar **errores**
 - Por ejemplo, si intentamos sumar una cadena y un entero:

```
'Hello' + 1  
Traceback (most recent call last):  
File "<stdin>", line 1, in <module>  
TypeError: can only concatenate str (not "int") to str
```

Tipos, operadores y expresiones

Expresiones

- Es posible combinar valores, variables y operadores para producir **expresiones**, que Python **evalúa** para producir un resultado
- Un tipo común de expresiones, las expresiones aritméticas, utilizan valores y operadores numéricos, y se parecen a fórmulas matemáticas:



Los operandos también pueden ser expresiones

- Si el intérprete se encuentra esta expresión, la evaluaría produciendo un resultado de **12**

Tipos, operadores y expresiones

Precedencia entre operadores

- Python utiliza **reglas de precedencia** bien definidas entre los distintos operadores para evaluar todo tipo de expresiones
- Por ejemplo, en expresiones aritméticas, las operaciones de multiplicación y división se calculan **antes** que las sumas y restas
- Los paréntesis se pueden utilizar para cambiar el orden de evaluación cuando es necesario
- El orden completo puede recordarse con la regla mnemotécnica ***papomudisure*** (**p**aréntesis, **p**otencia, **m**ultiplicación, **d**ivisión, **s**uma, **r**esta)

Tipos, operadores y expresiones

Expresiones: ejemplos

- **Ejemplo 1:** sea la siguiente expresión en Python:

```
7 + 5 * 4 // 3 + 5
```

- ¿Cuál es el resultado de esa expresión?
- Modifica la expresión mediante el uso de paréntesis para que el resultado sea 6

Tipos, operadores y expresiones

Expresiones: ejemplos

- **Ejemplo 2:** considera la siguiente expresión en Python::

```
3 * 1 ** 3
```

- ¿Cuál es el resultado?
 - A. 9
 - B. 3
 - C. 27
 - D. Ninguno de los anteriores

Tipos, operadores y expresiones

Operador de asignación (=)

- Para **definir una variable** se usa el operador de asignación (=) cuya sintaxis básica es:

```
variable = expresión
```

- Python evalúa la expresión y asigna el resultado a la variable
- Ejemplo:

```
var = 5
```

Se crea (*inicializa*) la variable var con un valor de 5

```
var = var + 1
```

Se incrementa en 1 el valor anterior y se reasigna a var (tras la instrucción, vale 6)

Tipos, operadores y expresiones

Operador de asignación (=)

- **Ejemplo:** considera la siguiente sentencia en Python:

```
x = a + 5 - b
```

- ¿Qué son, respectivamente, **a** y **b**, y **a + 5 - b**?
 1. Operandos y una expresión
 2. Operadores y una instrucción
 3. Instrucciones y una condición
 4. Operandos y una ecuación

Tipos, operadores y expresiones

Operador de asignación (=)

- Aumentar el valor de una variable es tan común, que Python tiene un operador que suma y reasigna un valor a una variable:

```
variable += expresión
```

- Es decir, `var = var + 1` se puede escribir como `var += 1`
- El ejemplo anterior se puede reescribir como:

```
var = 5  
var += 1
```

Tipos, operadores y expresiones

Operador de asignación (=)

- En realidad, todos los operadores aritméticos tienen su versión compacta correspondiente para el operador de asignación

Asignación	Valor de a
a = 1	1
a += 10	11
a -= 7	4
a *= 4	16
a /= 3	5
a %= 2	1
a /= 2	0.5
a **= 2	0.25

Python los denomina
operadores de
asignación
aumentados

Tipos, operadores y expresiones

Intercambio de valores entre 2 variables

- Si queremos **intercambiar** los valores almacenados en las variables a y b, el código habitual para hacerlo es:

<pre>a = 1 b = 2</pre>	←	Valores iniciales de las variables a y b
<pre>t = a</pre>	←	Asigna a t el valor de a (ahora ambas valen 1)
<pre>a = b</pre>	←	Asigna a a el valor de b (2) con lo que a pierde su antiguo valor (por eso lo hemos preservado en t)
<pre>b = t</pre>	←	Asigna a b el valor de t (1)

- Más adelante veremos que Python permite ejecutar este intercambio de manera más compacta y eficiente

Índice



- 1. Valores y variables**
- 2. Tipos, operadores y expresiones**
- 3. Tipos de datos básicos:
enteros, reales, cadenas y
booleanos**
- 4. Conversión de tipos**
- 5. Formatos de entrada y salida**

Tipos de datos básicos

Enteros (`int`)

- El tipo `int` es el que representa a los números enteros en Python
- A diferencia de otros lenguajes, Python puede representar sin problemas números enteros de cualquier tamaño
- Como hemos visto, los principales **operadores** de los enteros son:
 - `+`, `-` : suma, resta
 - `*` : multiplicación
 - `//` : división (entera)
 - `%` : módulo (resto de la división entera)
 - `**` : potencia

Tipos de datos básicos

Enteros: Ejemplos de expresiones aritméticas

Expresión entera	Valor	Descripción
+99	99	Signo positivo
-99	-99	Signo negativo
--99	99	Signo positivo
5 + 3	8	Suma
5 - 3	2	Resta
5 * 3	15	Multiplicación
5 // 3	1	Sin parte fraccionaria
5 % 3	2	Módulo
5 ** 3	125	Potencia
5 // 0	ZeroDivisionError	Error: División por cero
3 * 5 - 2	13	* tiene prioridad
3 + 5 // 2	5	// tiene prioridad
3 - 5 - 2	-4	Asociativo a la izquierda
(3 - 5) - 2	-4	Mejor estilo
3 - (5 - 2)	0	No ambiguo
2 ** 2 ** 3	256	Asociativo a la derecha
(2 ** 2) ** 3	64	Cambia la expresión
2 ** (2 ** 3)	256	Mejor estilo, no ambiguo

Tipos de datos básicos

Reales (float)

- El tipo **float** es el que representa a los números reales (o números en *coma flotante*) en Python
- En contra de lo que ocurre en matemáticas, en Python (y otros lenguajes) los números reales se almacenan con un tamaño y una precisión limitados, es decir, son **aproximaciones**
 - Se almacenan como fracciones de base 2 (binarias)
 - Por ejemplo: $0.1 + 0.1 + 0.1$ se evalúa como 0.30000000000000000004

Si quieres saber más:

<https://docs.python.org/3/tutorial/floatingpoint.html>

Tipos de datos básicos

Reales: Ejemplos de expresiones aritméticas reales

Expresión real	Valor	Descripción
3.141 + 2.0	5.141	Suma
3.141 - 2.0	1.141	Resta
3.141 * 2.0	6.282	Multiplicación
3.141 / 2.0	1.5705	División
5.0 / 3.0	1.6666666666666667	Los números reales tienen un máximo de 17 dígitos de precisión
3.141 ** 2.0	9.865881	Potencia
1.0 / 0.0	ZeroDivisionError	Error: División por cero
2.0 ** 10000.0	OverflowError	Error: No se puede representar un número real tan grande

Tipos de datos básicos

Cadenas (str)

- Las **cadenas** de texto o **strings** son secuencias de caracteres (letras, dígitos, símbolos, espacios y caracteres de control)
- Se representan entre comillas simples (' ') o dobles (" ")
- La barra invertida \ (escape) se usa para representar caracteres especiales dentro de la cadena:

Símbolo	Descripción	Ejemplo	Valor
\t	Tabulador	"Joe's\tcomputer"	'Joe's computer'
\n	Salto de línea	"Joe's\ncomputer"	'Joe's computer'
\'	Comilla simple	'Joe\'s computer'	'Joe's computer'
\\	Barra invertida	"Joe's\\ncomputer"	'Joe's\ncomputer'

Tipos de datos básicos

Cadenas: operadores

- **Suma** (`str1 + str2`): concatena dos cadenas `str1` y `str2`

Expresión	Valor
'Hello, ' + 'World'	'Hello, World'
'123' + '456'	'123456'
'1234' + ' ' + ' ' + '99'	'1234 + 99'
'123' + 456	TypeError : can only concatenate str (not "int") to str

- **Multiplicación** (`str1 * n`): suma `str1` consigo misma `n` veces

Expresión	Valor
'Hello' * 5	'HelloHelloHelloHelloHello'
20 * '_'	'_________________'
6 * '=' + 3 * '(0)' + 6 * '='	'=====(0)(0)(0)===='

Tipos de datos básicos

Cadenas: funciones

- Python proporciona multitud de **funciones** que operan sobre (o devuelven) cadenas. Algunos ejemplos:

Función	Descripción	Ejemplo
<code>len(s)</code>	Devuelve el tamaño de la cadena <code>s</code>	<pre>>>> len('Hi Py!') 6</pre>
<code>ord(c)</code>	Devuelve el código ASCII* de un carácter <code>c</code>	<pre>>>> ord('a') 97</pre>
<code>chr(code)</code>	Devuelve el carácter asociado a un código ASCII <code>code</code>	<pre>>>> chr(65) A</pre>

* <https://www.ascii-code.com/>

Tipos de datos básicos

Cadenas: métodos

- Así mismo, existen muchos **métodos*** aplicables sobre cadenas, por ejemplo:

Método	Descripción	Ejemplo (s = 'Valencia')
s.count(t)	Número de ocurrencias de la cadena t dentro de s	>>> s.count('a') 2
s.find(t)	Posición de la 1ª aparición de t en s. Si no es así, devuelve -1	>>> s.find('en') 3 >>> s.find('Pa') -1
s.upper()	Devuelve una copia en mayúsculas de s	>>> s.upper() 'VALENCIA'
s.lower()	Devuelve una copia en minúsculas de s	>>> s.lower() 'valencia'

***NOTA:** Más adelante explicaremos la diferencia entre método y función

Tipos de datos básicos

Cadenas: métodos

- Otros métodos:

Método	Descripción	Ejemplo (s = 'Valencia')
s.startswith(t)	Indica si s comienza con t	>>> s.startswith('Val') True
s.endswith(t)	Indica que s termina con t	>>> s.endswith('ia') True
s.replace(u, v)	Devuelve una copia de s con todas las apariciones de u reemplazadas por v	>>> s.replace('a', 'oo') 'Voolencioo'
s.isdigit()	Devuelve True si s consta solo de dígitos del 0 al 9	>>> 'Hello!'.isdigit() False >>> '123'.isdigit() True

Tipos de datos básicos

Cadenas: indexación

- Las cadenas son **secuencias** de caracteres, y cada carácter en ella ocupa una determinada **posición**
- En Python, las cadenas (y otros tipos de secuencias) se indexan comenzando en la **posición (o índice) 0**
- Por ejemplo, la cadena "Hello, World!" se indexaría así:

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12

↑
Primera
posición

↑
Última
posición

Tipos de datos básicos

Cadenas: indexación

- El **operador de indexación** (`[ ]`) se utiliza para acceder a cualquier carácter de la cadena mediante su **índice**. Su sintaxis es:

`cadena[índice]`

- Por ejemplo:

```
>>> var = 'Hello, World!'
>>> var[0]
'H'
>>> var[5]
','
>>> var[6]
' '
```

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12

Tipos de datos básicos

Cadenas: indexación

- Para cualquier cadena `s`, su **último** carácter es `s[len(s)-1]`
 - La última posición es la longitud de la cadena (`len(s)`) menos 1
 - Este es por tanto el último índice **posible** (válido) para esa cadena
- Al acceder a una cadena, es vital que el índice sea siempre **válido**
- Si usamos un índice “fuera de rango” (más allá de la última posición), Python lanza un **error** (excepción **IndexError**):

```
>>> var = 'Hello, World!'
```

```
>>> var[len(var) - 1]
```

```
'!'
```

```
>>> var[15]
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module> IndexError: string index out of range
```

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12

Tipos de datos básicos

Cadenas: indexación

- Python también admite indexar cadenas con **índices negativos**
- Para una cadena s , el **índice -1** corresponde a la última posición de la cadena, y el **índice $-\text{len}(s)$** a la primera posición
- Por ejemplo, si $s = \text{"Hello, World!"}$:

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

↑
 $s[0] = s[-\text{len}(s)]$

↑
 $s[\text{len}(s)-1] = s[-1]$

Tipos de datos básicos

Cadenas: indexación

- Con índices negativos también es necesario usar índices dentro del rango (de las posiciones correctas) de la cadena
- Cualquier acceso fuera de rango produce un error en ejecución
- Por ejemplo:

```
>>> var = 'Hello, World!'
```

```
>>> var[-1]
```

```
'!'
```

```
>>> var[-8]
```

```
','
```

```
>>> var[-14]
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module> IndexError: string index out of range
```

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Tipos de datos básicos

Cadenas: corte (*slicing*)

- Python permite extraer un **segmento** de la cadena mediante el **operador de corte** o **slicing**, que incluye **2 índices** entre los corchetes de indexación (inicio y fin):

`cadena[inicio:fin]`

y devuelve el segmento entre `cadena[inicio]` y `cadena[fin-1]`

- Ambos índices pueden ser **positivos** o **negativos**

```
>>> var = 'Hello, World!'
>>> var[2:5]
' llo'
>>> var[-7:-1]
' World'
```

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Tipos de datos básicos

Cadenas: corte (*slicing*)

- También podemos dejar **vacío** el primer o el último índice, lo que indica que el corte empieza por el primer carácter o acaba en el último
 - Si se omiten ambos índices, se incluye toda la cadena
- Por ejemplo:

```
>>> var = 'Hello, World!'
>>> var[5:]
', World!'
>>> var[: -9]
'Hell'
>>> var[:]
'Hello, World!'
```

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Tipos de datos básicos

Cadenas: corte (*slicing*)

- Finalmente, de manera opcional se puede incluir un **tercer índice**, que indica el **incremento** (del índice) y la **dirección** de recorrido:

```
cadena[inicio:fin:inc]
```

- Este valor **inc** indica:
 - Si es **> 1**, cuánto se incrementa el índice para incluir el siguiente carácter en el segmento de respuesta (cada 2, cada 3, etc.)
 - Si es un valor **negativo**, se invierte la dirección de recorrido, y se recorre la cadena de derecha a izquierda
 - En este caso el valor de **inicio** debe ser mayor que el de **fin**

Tipos de datos básicos

Cadenas: corte (*slicing*)

- Ejemplos:

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
>>> var = 'Hello, World!'
```

```
>>> var[5:10:2] ← Del 5 al 10-1, incluyendo 1 de cada 2 caracteres  
' ,Wr'
```

```
>>> var[::-1] ← Toda la cadena, en orden inverso  
'!dlroW ,olleH'
```

```
>>> var[1::-1] ← Del 1 al final (principio), en orden inverso  
'eH'
```

```
>>> var[1:3:-1] ← En orden inverso, si el inicio es menor que el fin no  
'' hay caracteres que incluir (cadena vacía)
```

Tipos de datos básicos

Booleanos (`bool`)

- El tipo **`bool`** es el que representa los valores de verdad en Python, y tiene dos posibles valores: **cierto** (**`True`**) y **falso** (**`False`**)
- Respecto a los **operadores** relacionados con este tipo, podemos diferenciar entre:
 - **Operadores lógicos**, que toman como argumentos valores booleanos y dan como resultado un valor booleano
 - **Operadores de comparación**, que comparan valores de cualquier tipo y cuyo resultado es un valor booleano
- A continuación, vamos a analizar ambos tipos de operadores

Tipos de datos básicos

Booleanos: operadores lógicos

- Hay 3 operadores lógicos: **and** (y), **or** (o) y **not** (no)
- Aplicados sobre variables booleanas a y b, sus **resultados** son:

a	b	a and b
False	False	False
False	True	False
True	False	False
True	True	True

a	b	a or b
False	False	False
False	True	True
True	False	True
True	True	True

a	not a
False	True
True	False

- **not** tiene prioridad sobre **and**, y este a su vez sobre **or**
- **and** no evalúa el segundo operando si el primero es **False**, y **or** no evalúa el segundo operando si el primero es **True**

Tipos de datos básicos

Booleanos: operadores de comparación

- Python ofrece **8 operadores de comparación** entre valores de cualquier tipo

Operador	Significado	Ejemplo (True)	Ejemplo (False)
<code>==</code>	Igual	<code>2 == (1 + 1)</code>	<code>2.0 == 3.5</code>
<code>!=</code>	Distinto	<code>5.0 != 2.5</code>	<code>(2 * 2) != 4.0</code>
<code><</code>	Menor	<code>2.5 < 5.0</code>	<code>5 < 5</code>
<code><=</code>	Menor o igual	<code>5 <= 5</code>	<code>5.0 <= 2.5</code>
<code>></code>	Mayor	<code>5.0 > 2.5</code>	<code>5 > 5</code>
<code>>=</code>	Mayor o igual	<code>5 >= 5</code>	<code>2.5 >= 5.0</code>
<code>is</code>	Mismo objeto	<code>5 is 5</code>	<code>5 is 5.0</code>
<code>is not</code>	Distinto objeto	<code>5 is not 5.0</code>	<code>5.0 is not 5.0</code>

Tipos de datos básicos

Booleanos: operadores de comparación

- Normalmente, valores de **tipos diferentes** se consideran siempre **distintos**, excepto valores numéricos que representan la misma magnitud, o los valores **True** y **False**, que equivalen a **1** y **0**

- Ejemplos:

```
>>> 1 == 1.0
True
>>> 1 == "1"
False
>>> 1 == True
True
>>> 0 == False
True
>>> 1 is 1.0
False
```

NOTA: 1.0 y 1 se consideran "iguales" (en magnitud), pero no representan el mismo objeto en memoria y por eso el operador `is` devuelve `False`

Tipos de datos básicos

Booleanos: operadores de comparación

- Las **comparaciones** (mayor que, menor que, etc.):
 - Entre valores **numéricos** (enteros o reales) se resuelven en función de la magnitud de los valores comparados
 - Entre **cadenas** s1 y s2 se sigue el **orden lexicográfico** (alfabético):
 - Se compara s1[0] y s2[0] según sus **códigos ASCII**
 - Si ambos son iguales, se sigue con el siguiente carácter, etc.

- Ejemplos:

```
>>> 'Hello' < 'hello'  ← Las mayúsculas tienen cód. ASCII menores ('H' < 'h')
True
>>> 'Hello' == 'hello' ← Mayúsculas y minúsculas son distintas
False
>>> 'Hello' > 'Hallo'  ← Se compara carácter a carácter ('e' > 'a')
True
```

Tipos de datos básicos

Valor nulo (None)

- Python incorpora un valor (*objeto*) explícito para denotar la ausencia de información, es decir, un valor “nulo” o inexistente
 - Es útil, por ejemplo, para saber si una función devuelve algún valor
- Este valor se denomina **None** (palabra reservada)
- **None** no es internamente 0, o una cadena vacía, sino que tiene un tipo propio, denominado `NoneType`

```
>>> None == 0
False
>>> type(None)
<class 'NoneType'>
```

Tipos de datos básicos

Resumen: precedencia de operadores

+ Precedencia ↑	Operador	Significado
	()	Paréntesis
	**	Potencia
	+ -	Signo (positivo, negativo)
	* / % //	Multiplicación, división, resto, div. entera
	+ -	Suma, resta
	<= < > >=	Comparación
	== !=	Igualdad
	= -= += *= %= /= // = **=	Asignación
	is is not	Identidad
	not or and	Lógicos

Índice



- 1. Valores y variables**
- 2. Tipos, operadores y expresiones**
- 3. Tipos de datos básicos: enteros, reales, cadenas y booleanos**
- 4. Conversión de tipos**
- 5. Formatos de entrada y salida**

Conversión de tipos

Conversión implícita

- Además del tipado dinámico, Python produce una **conversión de tipos implícita** en algunas ocasiones
- Al convertir el tipo de un valor a otro tipo **automáticamente**, se previenen errores sintácticos o pérdida de información
- El caso más habitual es la conversión de **enteros** en **reales**
- Ejemplos:

```
>>> 10 / 4
```

```
2.5
```

```
>>> 3 + 4.0
```

```
7.0
```

El resultado se convierte en un float para evitar pérdida de datos

3 se convierte en 3.0 (el valor con el tipo de menor precisión se “eleva” al tipo de mayor precisión)

Conversión de tipos

Conversión explícita

- Python ofrece funciones para convertir de manera **explícita** un valor de un tipo de datos en un valor de otro tipo, a criterio del programador
- Las funciones de **conversión** (**casting**) coinciden con los nombres de los tipos de datos correspondientes:
 - **int()**: convierte un valor (real, cadena,...) en **entero**
 - **float()**: convierte un valor (entero, cadena,...) en **real**
 - **bool()**: convierte un valor (de casi cualquier tipo) en **booleano**
 - **str()**: convierte un valor (real, entero,...) en **cadena**
- El valor que quiere convertirse debe ser adecuado

Conversión de tipos

Conversión explícita

Función	Ejemplos	Resultados	Descripción
int()	int(2.7) int(True) int('10') int('hola')	2 1 10 ValueError	Descartar decimales Verdadero = 1 y Falso = 0 Convertir cadena en entero Error: esa cadena no tiene equivalente entero
float()	float(2) float(True) float('2.5') float('2,5')	2.0 1.0 2.5 TypeError	De entero a número real Verdadero = 1.0 y Falso = 0.0 Convertir float en entero Error: la coma no es separador decimal
bool()	bool(0) bool(1.0) bool('True')	False True False	0 = Falso, cualquier otro entero = Verdadero 0.0 = Falso, cualquier otro real = Verdadero Cadena vacía = Falso, no vacía = Verdadero
str()	str(10) str(2.5) str(True)	'10' '2.5' 'True'	Convertir un entero en una cadena Convertir un real en cadena Convertir un booleano en cadena

Índice



- 1. Valores y variables**
- 2. Tipos, operadores y expresiones**
- 3. Tipos de datos básicos: enteros, reales, cadenas y booleanos**
- 4. Conversión de tipos**
- 5. Formatos de entrada y salida**

Formatos de entrada y salida

Entrada/salida de datos

- Para interactuar con el usuario y permitir la entrada y salida de datos, Python ofrece dos funciones principales: **input()** y **print()**
- **input()** se utiliza para **solicitar** por la consola información al usuario, que este introduce desde el teclado
 - La información introducida se almacena en una **variable**, que el programa puede utilizar posteriormente
- **print()** se utiliza para **imprimir** en la consola mensajes desde el programa
 - Puede imprimir directamente valores (y variables) de cualquier tipo, o bien una cadena formateada de manera específica

Formatos de entrada y salida

Entrada con `input()`

- El formato de esta función es el siguiente:

```
entrada = input( [mensaje] )
```

donde:

- `entrada`: variable de tipo **cadena** que recibe los datos tecleados por el usuario
- `mensaje`: cadena opcional que se muestra al usuario para informarle del dato que se solicita
- Al ejecutarse, el programa muestra el mensaje (si se incluye) y **espera** a que el usuario teclee su entrada y presione **Enter**

Formatos de entrada y salida

Entrada con `input()`

- **Ejemplo:** Entrada sin mensaje vs. entrada con mensaje:

```
>>> edad = input()  
18
```

```
>>> print(edad)  
18
```

```
>>> edad = input('Cuántos años tienes? ')  
¿Cuántos años tienes? 18
```

```
>>> print(edad)  
18
```

- Aunque es opcional, el mensaje se suele incluir **siempre**, para que el usuario sepa el motivo de que el programa se haya detenido en ese punto, y qué información se espera que introduzca

Formatos de entrada y salida

Entrada con `input()`

- Como **`input()`** siempre devuelve una cadena, si queremos que la entrada se **interprete** como un dato de otro tipo, debemos hacer una conversión explícita (***casting***)
- Ejemplo:

```
unidades = input('Dame un número:')  
Dame un número: 10  
>>> print('Resultado:', unidades, '* 2 =', unidades * 2)  
Resultado: 10 * 2 = 1010
```

← unidades es una cadena

```
unidades = int(input('Dame un número:'))  
Dame un número: 10  
>>> print('Resultado:', unidades, '* 2 =', unidades * 2)  
Resultado: 10 * 2 = 20
```

← Al usar **`int()`** unidades se convierte a entero

Formatos de entrada y salida

Salida con `print()`

- **print()** imprime información por pantalla (consola). Su sintaxis es:

```
print(*objetos, sep=' ', end='\n',...)
```

donde:

- **objetos**: datos de cualquier tipo, separados por comas. Se convierten implícitamente a tipo cadena, si no lo son ya
- **sep**: cadena que se usa para separar cada uno de los objetos (por defecto, **espacio** en blanco)
- **end**: cadena que se imprime al final de la secuencia de objetos (por defecto, un **salto** de línea)

Formatos de entrada y salida

Salida con print()

- Ejemplos:

```
print('Un coche azul')  
print('y', 'una casa', 'roja')
```

← end y sep por defecto



```
Un coche azul  
y una casa roja
```

```
print('Un coche azul', end=' ')  
print('y', 'una casa', 'roja', sep='--')
```

← end se cambia por un espacio (no hay salto de línea) y sep por dos guiones



```
Un coche azul y--una casa--roja
```

Formatos de entrada y salida

Salida con `print()`

- Además de las comillas simples y dobles, Python también permite las **triples comillas** (`''' '''`) que ignora los saltos de línea como fin de línea para el intérprete, pero los incorpora a la cadena
- Ejemplo:

```
var = '''Un coche  
azul  
  
y una casa roja  
'''  
print(var)
```



```
Un coche  
azul  
  
y una casa roja
```

Formatos de entrada y salida

Formatos de salida

- Python ofrece varias alternativas para generar cadenas de manera **sofisticada**, normalmente para **imprimirlas**
- Es posible incorporar a la salida valores, variables o expresiones de distintos tipos y formatearlos con precisión
- Explicaremos brevemente 2 de esas alternativas:
 - ***f-strings*** (*Formatted String Literals*)
 - Método **format** (sobre cadenas)
- Más información:
<https://docs.python.org/3/tutorial/inputoutput.html>

Formatos de entrada y salida

f-strings

- Los *Formatted String Literals*, o ***f-strings***, proporcionan una forma concisa y cómoda de incrustar expresiones dentro de cadenas
 - Se introdujeron en Python 3.6
- Para crear un *f-string*:
 - Se escribe el literal de cadena con la letra **f** o **F** **antes** de las comillas de apertura (por ejemplo, **f'...**)
 - En la cadena, se puede incluir **expresiones** entre llaves { } que se evaluarán en tiempo de ejecución y se formatearán mediante la función **str()** (conversión implícita a cadena)

Formatos de entrada y salida

f-strings

- Ejemplo:

```
>>> nombre = "Alice"
>>> edad = 30
>>> saludo = f"Hola, mi nombre es {nombre} y tengo {edad} años."
>>> print(saludo)
Hola, mi nombre es Alice y tengo 30 años.

>>> print(f"En 10 años, tendré {edad + 10} años.")
En diez años, tendré 40 años.
```

Con *f-strings* se pueden incorporar variables (expresiones) al crear la cadena (p.ej., `saludo`), o directamente en el `print()`

Formatos de entrada y salida

Método `str.format()`

- El método `format()` permite formatear de manera precisa ciertos valores que se incorporan a la cadena. Su sintaxis es:

```
cadena.format(*argumentos)
```

- donde:
 - `cadena`: puede contener **campos** (marcados entre `{}`) en los que se imprimirán ciertos argumentos (valores). Cada campo puede especificar un formato de visualización particular
 - `argumentos`: lista de **argumentos** (valores, expresiones, etc.), que se ubicarán en los campos respectivos de la cadena, y se visualizarán con las opciones de formato especificadas en ella

Formatos de entrada y salida

Método `str.format()`

- **Ejemplo:** correspondencia entre cada campo y su argumento:

```
>>> '{} {}, {}'.format('a', 'b', 'c')  ← Correspondencia por orden
'a, b, c'
>>> '{0}, {1}, {2}'.format('a', 'b', 'c') ← Correspondencia por índice
'a, b, c'                                     (comenzando en 0)
>>> '{2}, {1}, {0}'.format('a', 'b', 'c')
'c, b, a'
>>> '{0}{1}{0}'.format('abra', 'cad') ← Los índices pueden repetirse
'abracadabra'
```

Formatos de entrada y salida

Formato de *f-strings* y `str.format()`

- El **formato** de visualización de las expresiones es muy similar en ambas alternativas:

f-strings: `{[var]:[flags][width][.precision][type code]}`

`format()`: `{[pos]:[flags][width][.precision][type code]}`

donde:

- `var`: nombre de la variable (o expresión) que hay que formatear
- `pos`: posición de cada objeto de `format()`, comenzando por 0
- El resto de los parámetros son comunes (a continuación)

Formatos de entrada y salida

Formato de *f-strings* y `str.format()`

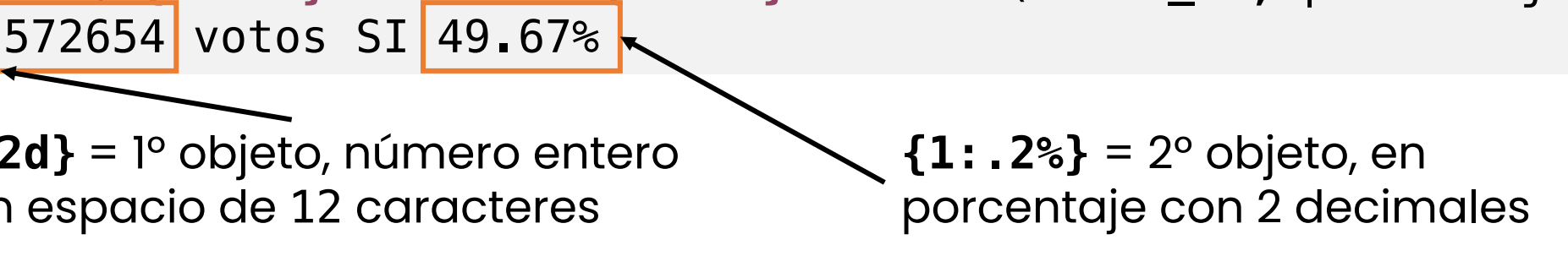
- El resto de los **parámetros** de visualización son:
 - **flags**: por ejemplo, para la alineación a la izquierda (<), derecha (>), centro (^), etc.
 - **width**: el tamaño mínimo para visualizar el valor correspondiente. Cuando el tamaño del valor es menor que este mínimo, los caracteres que faltan se rellenarán con espacios
 - **.precision**: el número de números decimales para representar un número flotante
 - **type code**: carácter que indica el tipo de datos:
 - **d** (entero decimal), **f** (real con decimales), **o** (entero octal), **x/X** (entero hexadecimal con letras minúsculas/mayúsculas), **b** (binario), **c** (carácter, convierte un valor entero en un Unicode), etc.

Formatos de entrada y salida

Formato de *f-strings* y `str.format()`

- **Ejemplo:** formato de visualización de cada campo/expresión:

```
>>> votos_si = 42572654
>>> votos_no = 43132495
>>> porcentaje = votos_si / (votos_si + votos_no)
>>> print('{0:12d} votos SI {1:.2%}'.format(votos_si, porcentaje))
42572654 votos SI 49.67%
```



`{0:12d}` = 1º objeto, número entero
en un espacio de 12 caracteres

`{1:.2%}` = 2º objeto, en
porcentaje con 2 decimales

```
>>> # Es equivalente a:
>>> print(f'{votos_si:12} votos SI {porcentaje:.2%}')
```

Formatos de entrada y salida

Formato de *f-strings* y `str.format()`

- **Ejemplo:** alineación de la salida:

<pre>>>> '{:<30}'.format('alineado a la izquierda') 'alineado a la izquierda'</pre>	← A la izquierda entre 30 espacios
<pre>>>> '{:>30}'.format('alineado a la derecha') ' alineado a la derecha'</pre>	← A la derecha entre 30 espacios
<pre>>>> '{:^30}'.format('centrado') ' centrado '</pre>	← Centrado entre 30 espacios
<pre>>>> '{:*^30}'.format('centrado') '*****centrado*****'</pre>	← Idem, con '*' como carácter de relleno

Formatos de entrada y salida

Formato de *f-strings* y `str.format()`

- **Ejemplo:** alternativas de especificación de valores/argumentos:

```
>>> print(f'Un ejemplo, float: {45.879:10.2f} y decimal {3:d}')  
Un ejemplo, float:      45.88 y decimal 3
```

```
>>> real = 45.879  
>>> decimal = 3  
>>> print(f'Un ejemplo, float: {real:10.2f} y decimal {decimal:d}')  
Un ejemplo, float:      45.88 y decimal 3
```

```
>>> print('Un ejemplo, float: {0:10.2f} y decimal {1:d}'.format(45.879, 3))  
Un ejemplo, float:      45.88 y decimal 3
```

```
>>> print('Un ejemplo, float: {n1:10.2f} y decimal {n2:d}'.format(n1=45.879, n2=3))  
Un ejemplo, float:      45.88 y decimal 3
```